

COURSE CODE/TITLE:	CPE010	SECTION:	CPE21S1
DATE PERFORMED:	9/7/26	DATE SUBMITTED:	13/7/26
NAME:	Duenas Ralph Florenz	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. ____ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A:

- 1.

```
#include <iostream>

int SumofTwoNum(int num1, int num2);

int main() {
    std::cout << SumofTwoNum(10,20) << std::endl;

    return 0;
}

int SumofTwoNum(int fNum, int lNum) {
    int total = fNum + lNum;
    return total;
}
```

Header File Declaration Section

```
1 #include <iostream>
```

Global Declaration Section

```
int SumofTwoNum(int num1, int num2);
```

Class Declaration and
Method Definition Section

```
int SumofTwoNum(int fNum, int lNum) {
    int total = fNum + lNum;
    return total;
}
```

Main Function

```
int main() {
    std::cout << SumofTwoNum(10,20) << std::endl;

    return 0;
}
```

Method Definition

```
int SumofTwoNum(int fNum, int lNum) {
    int total = fNum + lNum;
    return total;
}
```

- 2.

```

22 #include <iostream>
23
24 int main() {
25
26     int A;
27     int B;
28
29     std::cout << "Input the value of A: ";
30     std::cin >> A;
31
32     std::cout << "Input the value of B: ";
33     std::cin >> B;
34
35     if (A > B) {
36         std::cout << "A is greater than B" << std::endl;
37     }
38     else if (B > A) {
39         std::cout << "B is greater than A" << std::endl;
40     }
41     else {
42         std::cout << "A and B are equal" << std::endl;
43     }
44
45     return 0;
46 }

```

Header File Declaration Section

```
#include <iostream>
```

- 3.

```

# include <iostream>

bool logicalOperations (bool A, bool B);

int main(){
    bool T,F;

    std:: cout << "Input first boolean value (0 or 1): " ;
    std:: cin >> T;

    std:: cout << "Input second boolean value (0 or 1): ";
    std:: cin >> F;

    if (logicalOperations (T,F)){
        std:: cout << "SUCCESSFUL!";
    }
    return 0;
}

bool logicalOperations (bool A, bool B){
    std:: cout << "Logical Operations: " << std:: endl;
    std:: cout << "A and B : " << (A && B) << std:: endl;
    std:: cout << "A or B : " << (A || B) << std:: endl;
    std:: cout << "NOT A : " << (!A) << std:: endl;
    std:: cout << "NOT B : " << (!B) << std:: endl;

    return true;
}

```

Header File Declaration Section

```
#include <iostream>
```

Global Declaration Section

```
bool logicalOperations (bool A, bool B);
```

Class Declaration and Method Definition Section

```
int main(){
    bool T,F;

    std::cout << "Input first boolean value (0 or 1): " ;
    std::cin >> T;

    std::cout << "Input second boolean value (0 or 1): ";
    std::cin >> F;

    if (logicalOperations (T,F)){
        std::cout << "SUCCESSFUL!";
    }
    return 0;
}
```

Main Function

```
int main(){
    bool T,F;

    std::cout << "Input first boolean value (0 or 1): " ;
    std::cin >> T;

    std::cout << "Input second boolean value (0 or 1): ";
    std::cin >> F;

    if (logicalOperations (T,F)){
        std::cout << "SUCCESSFUL!";
    }
    return 0;
}
```

Method Definition

```
bool logicalOperations (bool A, bool B){
    std::cout << "Logical Operations: " << std::endl;
    std::cout << "A and B : " << (A && B) << std::endl;
    std::cout << "A or B : " << (A || B) << std::endl;
    std::cout << "NOT A : " << (!A) << std::endl;
    std::cout << "NOT B : " << (!B) << std::endl;

    return true;
}
```

ILO B:

- 1.

```

1 #include <iostream>
2
3 class Triangle {
4 private:
5     double totalAngle, angleA, angleB, angleC;
6
7 public:
8     Triangle(double A, double B, double C);
9     void setAngles(double A, double B, double C);
10    bool validateTriangle();
11 };
12
13 Triangle::Triangle(double A, double B, double C) {
14     angleA = A;
15     angleB = B;
16     angleC = C;
17     totalAngle = A + B + C;
18 }
19
20 void Triangle::setAngles(double A, double B, double C) {
21     angleA = A;
22     angleB = B;
23     angleC = C;
24     totalAngle = A + B + C;
25 }
26
27 bool Triangle::validateTriangle() {
28     return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);
29 }
30
31 int main() {
32
33     Triangle set1(40, 30, 110);
34
35     if (set1.validateTriangle()) {
36         std::cout << "The shape is a valid triangle.\n";
37     } else {
38         std::cout << "The shape is NOT a valid triangle.\n";
39     }
40
41     return 0;
42 }

```

The shape is a valid triangle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

```

1 #include <iostream>
2
3 int main() {
4     int num1, num2, ralph;
5
6     std::cout << "Enter first number: ";
7     std::cin >> num1;
8     std::cout << "Enter second number: ";
9     std::cin >> num2;
10
11     std::cout << "Before swapping: num1 = " << num1 << ", num2 = " << num2 << std::endl;
12
13     ralph = num1;
14     num1 = num2;
15     num2 = ralph;
16
17     std::cout << "After swapping: num1 = " << num1 << ", num2 = " << num2 << std::endl;
18
19     return 0;
20 }

```

```

Enter first number: 29
Enter second number: 45
Before swapping: num1 = 29, num2 = 45
After swapping: num1 = 45, num2 = 29
PS C:\Users\TIPQC\Documents\Duenas CPE010>

```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```
#include <iostream>

double kelvinToFahrenheit(double kelvin) {
    double fahrenheit = (kelvin - 273.15) * 9.0 / 5.0 + 32.0;
    return fahrenheit;
}

int main() {
    double kVal;

    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kVal;

    double fValue = kelvinToFahrenheit(kVal);

    std::cout << kVal << " Kelvin's value is: " << fValue << " Fahrenheit." << std::endl;

    return 0;
}
```

```
Enter temperature in Kelvin: 198
198 Kelvin's value is: -117.67 Fahrenheit.
```

3. Create a C++ program that has a function that will calculate the distance between two points.

```
1  #include <iostream>
2  #include <cmath>
3
4  double calculateDistance(double x1, double y1, double x2, double y2);
5
6  int main() {
7
8      double x1, y1, x2, y2;
9
10     std::cout << "Enter x1: ";
11     std::cin >> x1;
12
13     std::cout << "Enter y1: ";
14     std::cin >> y1;
15
16     std::cout << "Enter x2: ";
17     std::cin >> x2;
18
19     std::cout << "Enter y2: ";
20     std::cin >> y2;
21
22     double distance = calculateDistance(x1, y1, x2, y2);
23
24     std::cout << "Distance between the two points: " << distance << std::endl;
25
26     return 0;
27 }
28
29 double calculateDistance(double x1, double y1, double x2, double y2) {
30
31     return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
32
33 }
```

```
Enter x1: 14
Enter y1: 10
Enter x2: 12
Enter y2: 13
Distance between the two points: 3.60555
```

4. Modify the code given in ILO B and add the following functions:

a. A function to compute for the area of a triangle

```
#include <iostream>
#include <cmath>

class Triangle {
private:
    double sideA, sideB, sideC;
public:
    Triangle(double A, double B, double C);
    void setSides(double A, double B, double C);
    bool validateTriangle();
    double computeArea();
};

Triangle::Triangle(double A, double B, double C) {
    sideA = A;
    sideB = B;
    sideC = C;
}

void Triangle::setSides(double A, double B, double C) {
    sideA = A;
    sideB = B;
    sideC = C;
}

bool Triangle::validateTriangle() {
    return (sideA + sideB > sideC &&
            sideA + sideC > sideB &&
            sideB + sideC > sideA);
}

double Triangle::computeArea() {
    double s = (sideA + sideB + sideC) / 2;
    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

int main() {
    Triangle set1(3, 4, 5);

    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
        std::cout << "Area = " << set1.computeArea() << std::endl;
    }
    else{
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

The shape is a valid triangle.
Area = 6

b. A function to compute for the perimeter of a triangle

```
57 #include <iostream>
58
59 class Triangle {
60 private:
61     double sideA, sideB, sideC;
62
63 public:
64     Triangle(double A, double B, double C);
65     void setSides(double A, double B, double C);
66     bool validateTriangle();
67     double computePerimeter();
68 };
69
70 Triangle::Triangle(double A, double B, double C) {
71     sideA = A;
72     sideB = B;
73     sideC = C;
74 }
75
76 void Triangle::setSides(double A, double B, double C) {
77     sideA = A;
78     sideB = B;
79     sideC = C;
80 }
81
82 bool Triangle::validateTriangle() {
83     return (sideA + sideB > sideC &&
84         sideA + sideC > sideB &&
85         sideB + sideC > sideA);
86 }
87
88 double Triangle::computePerimeter() {
89     return sideA + sideB + sideC;
90 }
91
92 int main() {
93     Triangle set1(3,4,5);
94
95     if(set1.validateTriangle()){
96         std::cout << "The shape is a valid triangle.\n";
97         std::cout << "Perimeter = " << set1.computePerimeter() << std::endl;
98     }
99     else{
100         std::cout << "The shape is NOT a valid triangle.\n";
101     }
102 }
103
104 return 0;
105 }
```

The shape is a valid triangle.
Perimeter = 12

c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
#include <iostream>
#include <algorithm>

class Triangle {
private:
    double sideA, sideB, sideC;
public:
    Triangle(double A, double B, double C);
    void setSides(double A, double B, double C);
    bool validateTriangle();
    void determineTriangleType();
};

Triangle::Triangle(double A, double B, double C) {
    sideA = A;
    sideB = B;
    sideC = C;
}

void Triangle::setSides(double A, double B, double C) {
    sideA = A;
    sideB = B;
    sideC = C;
}

bool Triangle::validateTriangle() {
    return (sideA + sideB > sideC &&
            sideA + sideC > sideB &&
            sideB + sideC > sideA);
}

void Triangle::determineTriangleType() {
    double a = sideA;
    double b = sideB;
    double c = sideC;

    if(a > c) std::swap(a,c);
    if(b > c) std::swap(b,c);

    if((a*a + b*b) > (c*c))
        std::cout << "Triangle is Acute-angled.\n";
    else if((a*a + b*b) < (c*c))
        std::cout << "Triangle is Obtuse-angled.\n";
    else
        std::cout << "Triangle is Right-angled (Others).\n";
}

int main() {
    Triangle set1(3,4,5);

    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
        set1.determineTriangleType();
    }
    else{
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

The shape is a valid triangle.
Triangle is Right-angled (Others).

- CONCLUSION

Summary of lessons learned

- I learned how to create and use functions in C++. I also learned how to organize a program into different sections and call functions before or from the main function. These lessons helped me code cleaner and more organized code.

Analysis of the procedure

- The procedure was easy to follow because it was done step by step. It helped me understand how each part of the program works before combining them into one program. By following the instructions, I was able to complete the required tasks.

Analysis of the supplementary activity

- The supplementary activity gave me more practice in using functions to solve different programming problems. It improved my logical thinking and coding skills by applying the concepts to real examples. It also helped me become more confident in coding C++.

Concluding Statement / Feedback

- I think I did well in this HOA because I was able to complete the programs and understand their purpose. I still need to improve my coding speed and reduce small mistakes in my usage of variables and thinking critically. With more practice, I believe I can become better at programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Idea generation , Explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.